06 DNS Sinkhole & Domain Filter

06-dns-sinkhole

A dnslib DNS resolver that sinkholes ad/tracker/malware domains from the live StevenBlack list, forwards the rest upstream, and logs every query to SQLite with a Streamlit dashboard.

Verified here: 52 passed; 0 failed; 0 skipped

Measured results

- Live StevenBlack blocklist fetched this run: 74,480 domains (bundled 2,006-domain fallback when offline)
- 400/400 seeded loopback queries got the expected block/allow/forward decision, including label-boundary look-alikes
- Blocked names never reached upstream: 238 of 400 queries forwarded, 162 answered locally
- Dead upstream: 10/10 queries returned SERVFAIL in avg 501.7 ms (500 ms timeout); blocking kept working
- Policy matcher: ~460k lookups/s (2.2 us each) on the full list
- Live smoke test via 1.1.1.1: real names resolved, ad.doubleclick.net sinkholed to 0.0.0.0 / ::

Data provenance and caveats

- StevenBlack unified hosts https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts (MIT) - fetched live during this run (74,480 domains, list dated 26 Oct 2026)
- data/stevenblack_snapshot.txt: bundled fallback, an evenly spaced sample of 2,000 real entries + 6 pinned well-known ones
- Demo traffic: a generated, seeded query mix against a local stub upstream serving RFC 5737/3849 documentation IPs
- Live smoke test: 6 real queries forwarded to public resolver 1.1.1.1 from loopback
- Whether the live list or the fallback is used depends on network at run time; this run got the live list.
- Demo block rate and throughput come from a generated query mix on one machine with one sequential client, not real household traffic.
- No real network interception: the server binds 127.0.0.1 only; clients were pointed at it explicitly.
- No DNSSEC, DoH/DoT upstream or response cache (listed as future work). URLhaus/OpenPhish feeds not added; only StevenBlack.

Project contents

README.md, requirements.txt, cli.py, app.py, src/__init__.py, src/blocklist.py, src/core.py, src/demo.py, src/domains.py, src/querylog.py, src/resolver.py, src/server.py, tests/conftest.py, tests/test_blocklist.py, tests/test_domains.py, tests/test_querylog_cli.py, tests/test_server.py, .gitignore, pytest.ini, scripts/make_charts.py

DNS Sinkhole / Ad & Domain Filter

A working DNS filtering resolver in Python (dnslib): it answers ad/tracker/malware hostnames from the public StevenBlack blocklist with a sinkhole address, forwards everything else to an upstream resolver, logs every query to SQLite and shows the results in a Streamlit dashboard.

Problem

Every ad, tracker and a lot of malware has to look up a hostname before it can connect. A DNS sinkhole (the idea behind Pi-hole) sits between devices and the real resolver and refuses to resolve known-bad names, which blocks those connections for every device on the network without installing anything on them. The parts that matter to get right: matching domains correctly (no accidental over-blocking), letting the user fix false positives with an allowlist, and failing safely when the upstream resolver or the blocklist download is unavailable.

How it works / Architecture

```
  client (dig / browser / cli.py query)
        |  UDP or TCP, 127.0.0.1:53535
        v
  +------------------------ SinkholeResolver -------------------------+
  |  normalise qname --> DomainPolicy.decide()                         |
  |                      1. allowlist match?  -> forward  ("allowed") |
  |                      2. blocklist match?  -> sinkhole ("blocked") |
  |                      3. otherwise         -> forward  ("forwarded")|
  |  sinkhole: A -> 0.0.0.0, AAAA -> ::, other types -> NXDOMAIN, TTL 60 |
  |  forward : UDP to upstream with timeout (TCP retry if truncated)    |
  |            timeout / error / id mismatch -> SERVFAIL ("servfail")   |
  +----------------------------|---------------------------------------+
                               v
                SQLite query log (ts, qname, qtype, action, rule, rcode, latency)
                               |
                cli.py stats  /  Streamlit dashboard (app.py)

  Blocklist loader: live StevenBlack hosts (timeout + size cap)
                  -> last good cache -> bundled 2,006-domain snapshot
```

Modules:

file	role
`src/domains.py`	domain normalisation (lower-case, trailing dot, IDNA, label validation) and the rule engine: exact rules, suffix rules (`*.example.com`) matched on label boundaries, allowlist-overrides-blocklist policy
`src/blocklist.py`	hosts-file parser, live fetch with timeout and 20 MB cap, cache, bundled fallback snapshot
`src/resolver.py`	the dnslib `BaseResolver`: sinkhole answers, upstream forwarding, SERVFAIL fallback, logging
`src/querylog.py`	thread-safe SQLite log (WAL) and SQL statistics
`src/server.py`	start/stop helpers for dnslib's threaded UDP/TCP servers, a stub upstream for tests, a tiny DNS client
`src/demo.py`	reproducible loopback demo and matcher benchmark
`cli.py`	`fetch-blocklist`, `check`, `serve`, `query`, `stats`, `demo`
`app.py`	Streamlit dashboard: overview, query log, top domains, policy tester

Security design decisions

- **Matching on label boundaries, not string suffixes.** A suffix rule `*.ads.example` is checked by walking the query name's parents (`a.b.ads.example` -> `b.ads.example` -> `ads.example` -> `example`), so `badads.example`, `myads.example` and `ads.example.evil.test` are never matched by accident. Lookups cost O(number of labels): about 2 microseconds each against the 74k-domain list.
- **Hosts entries are exact by default.** StevenBlack lists exact hostnames; treating each as "domain + all subdomains" would over-block (e.g. CDN or cloud parents). `--hosts-mode suffix` exists for people who want the stricter behaviour; wildcard rules go in `data/custom_blocklist.txt`.
- **Allowlist always wins.** False positives are the main operational cost of DNS blocking, so an allow rule (exact or suffix) overrides any block rule. The bundled allowlist re-enables `www.googleadservices.com`, a real StevenBlack entry that breaks Google Shopping links.
- **Fail closed to a definite answer, not a hang.** Upstream timeouts (default 2 s, 0.5 s in tests), refused ports, socket errors, unparseable replies and reply-ID mismatches all become an immediate SERVFAIL so clients fall back quickly. Blocked names are answered locally, so blocking keeps working while upstream is down.
- **Blocklist download is untrusted input.** Fetch has a timeout and a 20 MB size cap, only lines mapping to sink addresses (0.0.0.0, 127.0.0.1, ::, ::1) are accepted, every hostname is validated, local names (localhost, broadcasthost...) are skipped, and a near-empty download is rejected rather than silently disabling the filter. The cache is written atomically.
- **Sinkhole address 0.0.0.0 / ::** rather than a local web server: the connection fails immediately and no traffic reaches any host. Blocked answers use a short TTL (60 s) so un-blocking takes effect quickly.
- **Safe defaults.** The server binds `127.0.0.1:53535` (no root, no clash with mDNS on 5353). Binding a LAN interface requires `--allow-non-loopback`. Nothing here changes system resolver settings or intercepts traffic; you point a client at it explicitly.
- **Privacy.** A DNS log is a browsing history. `--no-client-ip` stops client IPs being stored; the demo log is written with client IPs off.
- **Upstream is an IP literal** (no bootstrap DNS lookup, no way for a name to be hijacked before the resolver starts).

Threat model: protects users on a network from resolving known ad/tracking/malware hostnames. It does not stop software that uses hard-coded IPs, DNS-over-HTTPS/TLS to its own resolver, or names not yet on the list; plain UDP DNS to the upstream can be observed and spoofed on path (see Limitations).

Project structure

```
06-dns-sinkhole/
??? cli.py                 command-line interface
??? app.py                 Streamlit dashboard
??? src/
?   ??? domains.py         normalisation + rule engine + policy
?   ??? blocklist.py       hosts parsing, live fetch, fallback
?   ??? resolver.py        dnslib resolver (sinkhole / forward / SERVFAIL)
?   ??? querylog.py        SQLite log + stats
?   ??? server.py          server helpers, stub upstream, DNS client
?   ??? core.py            wiring
?   ??? demo.py            loopback demo + benchmark
??? data/
?   ??? stevenblack_snapshot.txt   bundled fallback (2,006 real entries, MIT licence)
?   ??? allowlist.txt
?   ??? custom_blocklist.txt
??? scripts/make_charts.py
??? tests/                 52 pytest tests (servers on random loopback ports)
??? outputs/               real run artifacts (logs, stats, charts)
```

```

## Setup & how to run

```bash
pip install -r requirements.txt

python cli.py fetch-blocklist                    # live StevenBlack list (falls back if
 offline)
python cli.py check ad.doubleclick.net notdoubleclick.net www.googleadservices.com

# run the sinkhole on loopback (Ctrl-C to stop), forwarding to Cloudflare
python cli.py serve --port 53535 --upstream 1.1.1.1 --db outputs/queries.db
# in another terminal
python cli.py query ad.doubleclick.net --type AAAA --port 53535
dig @127.0.0.1 -p 53535 example.com              # if dig is installed
python cli.py stats --db outputs/queries.db

python cli.py demo                               # self-contained demo: stub upstream +
 sinkhole + 400 queries
python scripts/make_charts.py

pytest -v                                        # 52 tests; 2 are marked network
streamlit run app.py
```

Results (measured on this run)

- **Blocklist:** the live StevenBlack list was reachable during this run: **74,480
domains** parsed (upstream header: 74,480 unique domains, dated 26 Oct 2026); with the
custom rules that is 74,483 block rules plus 2 allow rules. Offline, the bundled snapshot
gives 2,006 domains. Which one you get depends on the network at run time, and `fetch-
blocklist`/`demo` print the source.
- **Tests:** 52 passed, 0 failed (`outputs/pytest_output.txt`), including 15 tests that
start real UDP/TCP DNS servers on random loopback ports, and 2 tests marked `network`
(live list fetch, real forwarding through 1.1.1.1), which both passed here.
- **Loopback demo** (stub upstream serving documentation IPs, seeded query mix): **400
queries, 400/400 got the expected decision** (blocked / allowed / forwarded, including
label-boundary look-alikes). The upstream received only 238 queries: blocked names never
left the resolver. Throughput 1011.6 queries/s over UDP loopback, single client,
sequential.
- **Query log totals** (418 rows): blocked 170, allowed 33, forwarded 205, servfail 10
(block rate 40.67%; the rate reflects the demo mix, not real traffic).
- **Latency inside the resolver:** sinkholed answers avg 0.05 ms; forwarded to the local
stub avg 0.44 ms; real forwarding to 1.1.1.1 in the live smoke test 6.2-12.6 ms.
- **Upstream failure:** 10/10 queries to a silent upstream returned SERVFAIL, avg 501.7 ms
 with a 500 ms timeout; a blocked name queried during the outage was still answered
`0.0.0.0`.
- **Matcher:** about 460,000 policy lookups/s (2.2 µs each) on the full list (pure Python,
 one core).
- **Live smoke test:** `cli.py serve` on 127.0.0.1:53535 forwarding to 1.1.1.1 answered 6
real queries: example.com A/AAAA resolved, ad.doubleclick.net A/AAAA sinkholed to 0.0.0.0
/ ::, the allowlisted www.googleadservices.com resolved, and doubleclick.net.example.org
was forwarded (not blocked).

Charts (`outputs/`): `chart_actions.png` (decision counts), `chart_latency.png` (latency
per decision, log scale), `chart_top_blocked.png` (most-queried blocked names in the demo
mix).

Sample output
```

```
$ python cli.py check ad.doubleclick.net notdoubleclick.net www.googleadservices.com
pixel.ads.example news.ads.example myads.example
# blocklist: live, 74,480 domains
ad.doubleclick.net                block  ad.doubleclick.net
notdoubleclick.net                pass
www.googleadservices.com                allow  www.googleadservices.com
pixel.ads.example                 block  *.ads.example
news.ads.example                    allow  news.ads.example
myads.example                     pass

$ python cli.py demo
sample answers (name, type, action, rcode, answer):
  ad.doubleclick.net          A     blocked    NOERROR   0.0.0.0   (rule
ad.doubleclick.net)
  ad.doubleclick.net          AAAA  blocked    NOERROR   ::   (rule ad.doubleclick.net)
  pixel.ads.example           A     blocked    NOERROR   0.0.0.0   (rule *.ads.example)
  news.ads.example            A     allowed    NOERROR   198.51.100.5   (rule
news.ads.example)
  myads.example               A     forwarded  NOERROR   198.51.100.6
  www.example.com             AAAA  forwarded  NOERROR   2001:db8::11
  no-such-host.example.net    A     forwarded  NXDOMAIN  -

query mix: 400 queries in 0.395s (1011.6 q/s over UDP loopback)
  action matched expectation: 400/400
  upstream received 238 queries (blocked names never leave the box)
failure test: 10 queries to a dead upstream -> SERVFAIL, avg 501.7 ms (timeout 500 ms)
  blocked name during outage still answered: 0.0.0.0

$ python cli.py query ad.doubleclick.net --type AAAA --port 53535     # live server,
upstream 1.1.1.1
status: NOERROR  id: 20235
  ad.doubleclick.net.  ttl=60  AAAA  ::

$ python cli.py fetch-blocklist --url https://10.255.255.1/hosts --timeout 1
source:   fallback  (data/stevenblack_snapshot.txt)
domains:  2,006
note:     live fetch not used -> ConnectTimeout: ... (connect timeout=1.0)
```

Full output: `outputs/sample_output.txt`.

Limitations & future work

- Plain UDP/TCP DNS to the upstream: no DNSSEC validation and no encrypted upstream (DoH/DoT), so an on-path attacker could see or spoof forwarded answers. Adding DoT/DoH upstream is the obvious next step.
- No response cache: every allowed query goes upstream. A TTL-respecting cache would cut latency and upstream load.
- Clients using their own DoH resolver or hard-coded IPs bypass any DNS filter.
- Only the StevenBlack list is fetched; URLhaus/OpenPhish-style feeds could be added through the same loader (they need per-feed parsers and licence checks).
- The demo traffic is a generated query mix against a stub upstream that serves RFC 5737/3849 documentation addresses. The block rate and throughput describe this run on this machine, not real household traffic. Throughput was measured with one sequential client.
- The blocklist is loaded at start-up; reloading without restart (SIGHUP) is future work.

Resume bullets

DNS Sinkhole & Domain Filter / README

- Built a DNS sinkhole resolver in Python (dnslib) that filters queries against the live 74,480-domain StevenBlack blocklist, answers blocked A/AAAA queries with 0.0.0.0/:: and forwards the rest upstream, with an allowlist that overrides block rules.
- Implemented label-boundary domain matching (exact + wildcard suffix rules) at about 460k lookups/s, so look-alikes such as `notdoubleclick.net` are never over-blocked; 400/400 queries in a seeded loopback test got the expected decision.
- Designed failure handling for upstream outages and blocklist download failures: timeouts return SERVFAIL in ~0.5 s, blocking keeps working during outages, and the loader falls back to a cached or bundled list.
- Logged every query to SQLite with a Streamlit + Plotly dashboard and a CLI; verified with 52 pytest tests, including integration tests that start real UDP/TCP DNS servers on random ports.

DNS Sinkhole & Domain Filter — ACTUAL SAMPLE OUTPUT

```
$ python cli.py fetch-blocklist
source:   live  (https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts)
domains:  74,480
elapsed:  0.17s

$ python cli.py fetch-blocklist --offline
source:   cache  (data/cache/stevenblack_hosts.txt)
domains:  74,480
elapsed:  0.10s
note:     live fetch not used -> offline mode requested

$ python cli.py check ad.doubleclick.net notdoubleclick.net www.googleadservices.com pixel.ads.example news.ads.example myads.example
# blocklist: live, 74,480 domains
ad.doubleclick.net                      block  ad.doubleclick.net
notdoubleclick.net                      pass
www.googleadservices.com                allow  www.googleadservices.com
pixel.ads.example                       block  *.ads.example
news.ads.example                        allow  news.ads.example
myads.example                           pass

$ python cli.py demo
blocklist: live -> 74,480 domains from https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts
policy: 74,483 block rules, 2 allow rules

sample answers (name, type, action, rcode, answer):
  ad.doubleclick.net           A     blocked    NOERROR   0.0.0.0   (rule ad.doubleclick.net)
  ad.doubleclick.net           AAAA  blocked    NOERROR   ::   (rule ad.doubleclick.net)
  pixel.ads.example            A     blocked    NOERROR   0.0.0.0   (rule *.ads.example)
  news.ads.example             A     allowed    NOERROR   198.51.100.5   (rule news.ads.example)
  myads.example                A     forwarded  NOERROR   198.51.100.6
  www.example.com              AAAA  forwarded  NOERROR   2001:db8::11
  no-such-host.example.net     A     forwarded  NXDOMAIN  -

query mix: 400 queries in 0.395s (1011.6 q/s over UDP loopback)
  action matched expectation: 400/400
  upstream received 238 queries (blocked names never leave the box)
failure test: 10 queries to a dead upstream -> SERVFAIL, avg 501.7 ms (timeout 500 ms)
  blocked name during outage still answered: 0.0.0.0

log totals: 418 queries, block rate 40.67%
  blocked      170   avg 0.05 ms
  allowed       33   avg 0.436 ms
  forwarded    205   avg 0.44 ms
  servfail      10   avg 500.653 ms

matcher: 463,460 policy lookups/s (2.16 us each)

$ python cli.py stats --db outputs/demo_queries.db --top 5
total queries: 418   block rate: 40.67%
  blocked       170   avg    0.05 ms   max     0.11 ms
  allowed        33   avg    0.44 ms   max     0.52 ms
  forwarded     205   avg    0.44 ms   max     2.34 ms
  servfail       10   avg  500.65 ms   max   500.77 ms
top blocked:
     47  smetrics.pearlevision.ca
     19  fe.brandreachsys.com
     13  affiliate.googleusercontent.com
     12  www.2n2qlh5hqcwrvo.net
      7  a.b.ads.example

# Live smoke test: real sinkhole on 127.0.0.1:53535 forwarding to a public resolver (1.1.1.1), stopped after 12 s
$ python cli.py serve --port 53535 --upstream 1.1.1.1 --db outputs/queries.db --duration 12
```

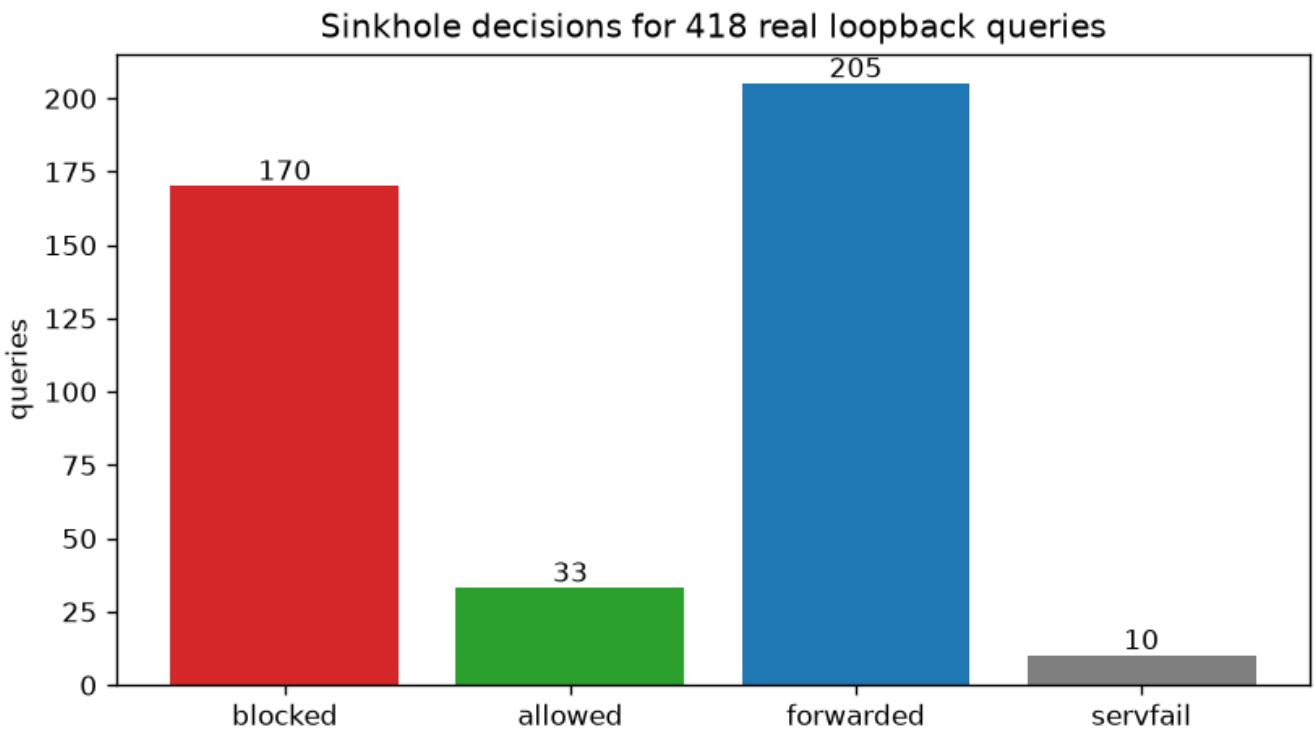
```
$ python cli.py query example.com --type A --port 53535
status: NOERROR  id: 26351
  example.com.  ttl=22  A  172.66.147.243
  example.com.  ttl=22  A  104.20.23.154
$ python cli.py query example.com --type AAAA --port 53535
status: NOERROR  id: 1127
  example.com.  ttl=226  AAAA  2606:4700:10::6814:179a
  example.com.  ttl=226  AAAA  2606:4700:10::ac42:93f3
$ python cli.py query ad.doubleclick.net --type A --port 53535
status: NOERROR  id: 30662
  ad.doubleclick.net.  ttl=60  A  0.0.0.0
$ python cli.py query ad.doubleclick.net --type AAAA --port 53535
status: NOERROR  id: 20235
  ad.doubleclick.net.  ttl=60  AAAA  ::
$ python cli.py query www.googleadservices.com --type A --port 53535
status: NOERROR  id: 11447
  www.googleadservices.com.  ttl=270  A  74.125.199.155
  www.googleadservices.com.  ttl=270  A  74.125.199.157
  www.googleadservices.com.  ttl=270  A  74.125.199.154
  www.googleadservices.com.  ttl=270  A  74.125.199.156
$ python cli.py query doubleclick.net.example.org --type A --port 53535
status: NOERROR  id: 14317
--- serve output ---
blocklist: live (74,480 domains); allowlist rules: 2
listening on 127.0.0.1:53535 (UDP+TCP) -> upstream 1.1.1.1:53; log outputs/queries.db
try: dig @127.0.0.1 -p 53535 ad.doubleclick.net   (Ctrl-C to stop)
{"blocked": 2, "allowed": 1, "forwarded": 3, "servfail": 0}

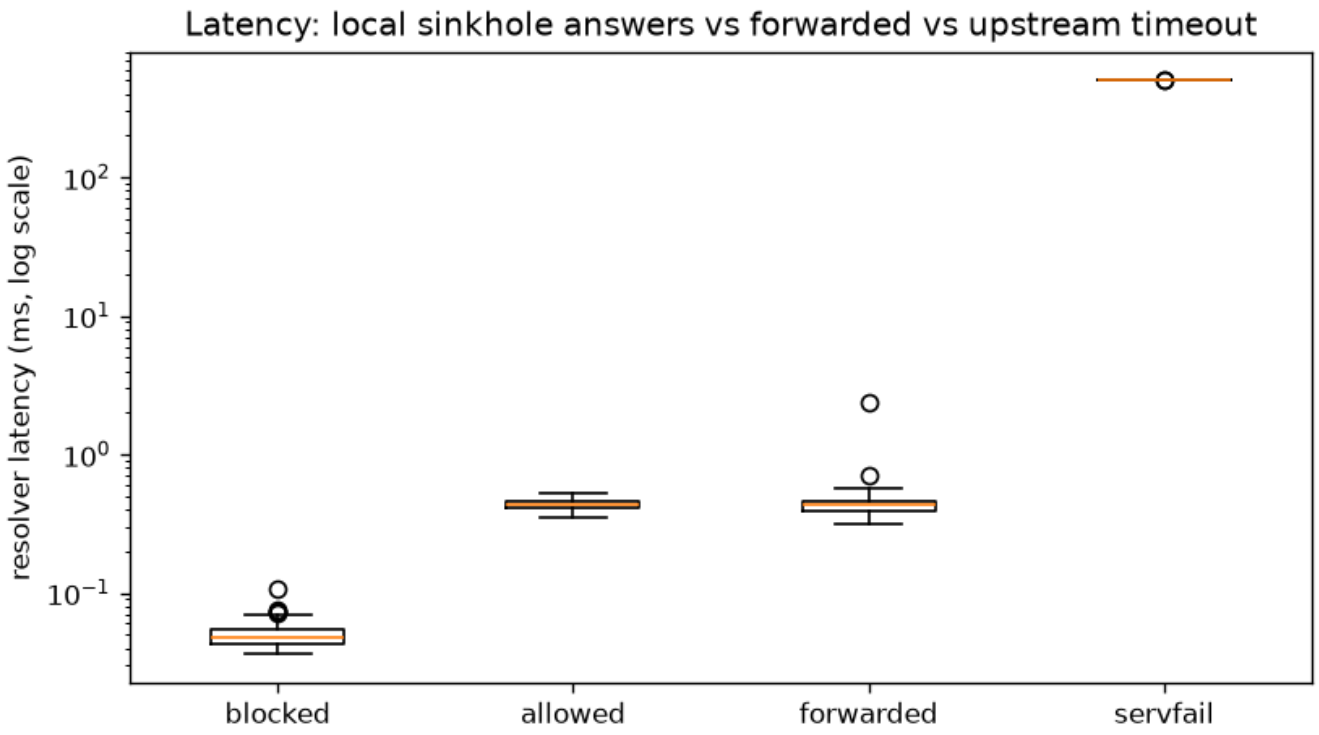
$ python cli.py stats --db outputs/queries.db
total queries: 6   block rate: 33.33%
  blocked         2   avg    0.09 ms   max     0.09 ms
  allowed         1   avg    6.20 ms   max     6.20 ms
  forwarded       3   avg    8.80 ms   max    12.62 ms
  servfail        0   avg    0.00 ms   max     0.00 ms
top blocked:
      2  ad.doubleclick.net

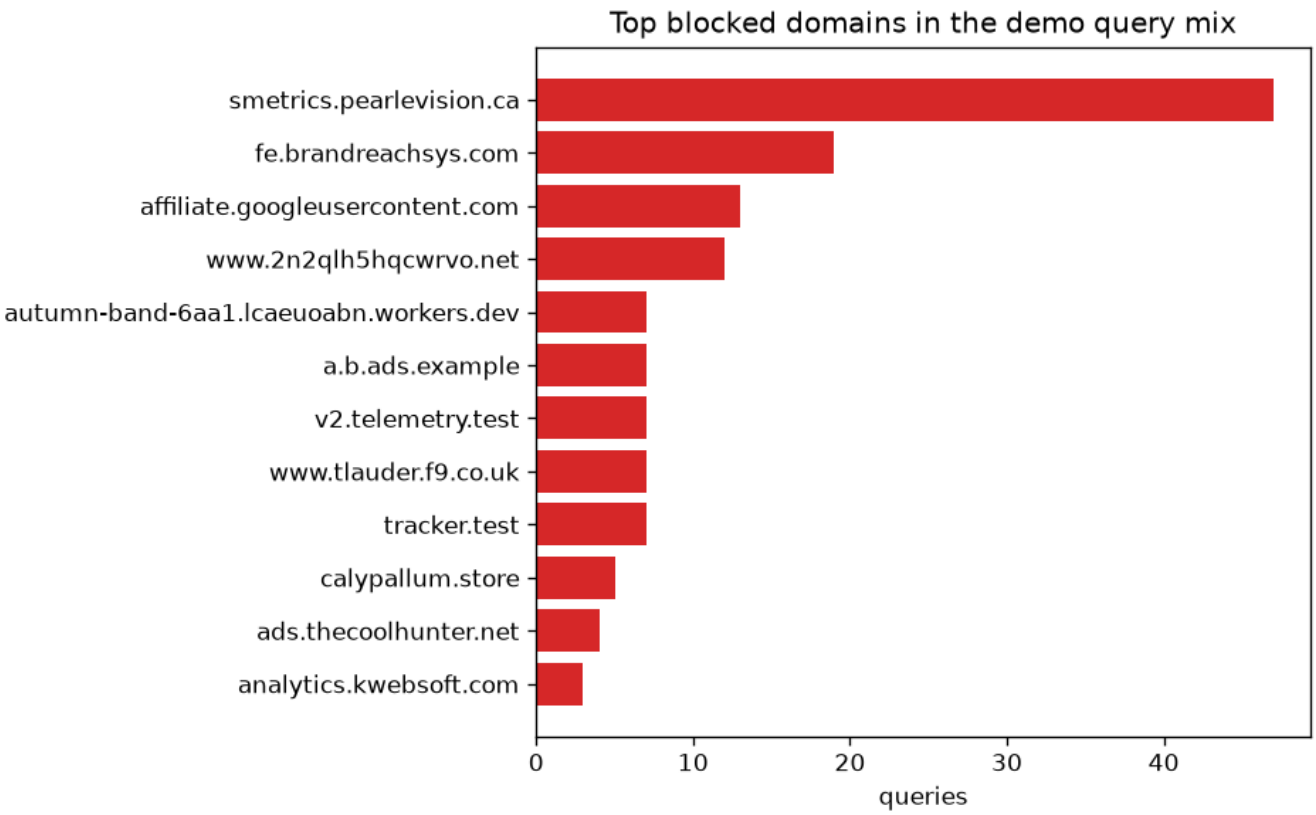
# Fallback behaviour: cache deleted, then (a) offline and (b) a live fetch that cannot finish in time
$ rm -rf data/cache && python cli.py fetch-blocklist --offline
source:   fallback  (data/stevenblack_snapshot.txt)
domains:  2,006
elapsed:  0.00s
note:     live fetch not used -> offline mode requested

$ python cli.py fetch-blocklist --url https://10.255.255.1/hosts --timeout 1
source:   fallback  (data/stevenblack_snapshot.txt)
domains:  2,006
elapsed:  1.01s
note:     live fetch not used -> ConnectTimeout: HTTPSConnectionPool(host='10.255.255.1', port=443): Max retries exceeded with url: /hosts (Caused by ConnectTimeoutError(<HTTPSConnection(host='10.255.255.1', port=443) at 0x7fd3d38ad590>, 'Connection to 10.255.255.1 timed out. (connect timeout=1.0)'))

```


Sinkhole decisions for 418 real loopback queries
queries
0
25
50
75
100
125
150
175
200
170
33
205
10
blocked
allowed
forwarded
servfail


Latency: local sinkhole answers vs forwarded vs upstream timeout
resolver latency (ms, log scale)
10^2
10^1
10^0
10^{-1}
blocked
allowed
forwarded
servfail


Top blocked domains in the demo query mix
smetrics.pearlevision.ca
fe.brandreachsys.com
affiliate.googleusercontent.com
www.2n2qlh5hqcwrvo.net
autumn-band-6aa1.lcaeuoabn.workers.dev
a.b.ads.example
v2.telemetry.test
www.tlauder.f9.co.uk
tracker.test
calypallum.store
ads.thecoolhunter.net
analytics.kwebsoft.com
0
10
20
30
40
queries

06-dns-sinkhole/requirements.txt

```
dnslib>=0.9.24
requests>=2.31
pandas>=2.0
plotly>=5.18
matplotlib>=3.8
streamlit>=1.33
pytest>=8.0
pytest-timeout>=2.2

```

06-dns-sinkhole/cli.py

```
#!/usr/bin/env python3
"""DNS Sinkhole command-line interface.

Examples:
    python cli.py fetch-blocklist                # live StevenBlack list, falls back to bundled s
napshot
    python cli.py check ad.doubleclick.net www.googleadservices.com
    python cli.py serve --port 53535 --upstream 1.1.1.1 --db outputs/queries.db
    python cli.py query ad.doubleclick.net --type AAAA --port 53535
    python cli.py stats --db outputs/queries.db
    python cli.py demo                           # self-contained loopback demo
"""
from __future__ import annotations

import argparse
import ipaddress
import json
import signal
import sys
import time
from pathlib import Path

from dnslib import QTYPE, RCODE

from src.blocklist import STEVENBLACK_URL, load_blocklist
from src.core import DEFAULT_ALLOWLIST, DEFAULT_CUSTOM_BLOCKLIST, build_policy, build_sinkhole
from src.querylog import stats_from_file
from src.resolver import ResolverConfig
from src.server import DEFAULT_HOST, DEFAULT_PORT, dns_query, start_dns_server

ROOT = Path(__file__).resolve().parent


def parse_upstream(value: str) -> tuple[str, int]:
    host, _, port = value.partition(":")
    ipaddress.ip_address(host)          # upstream must be an IP literal (no bootstrap DNS)
    return host, int(port or 53)


def cmd_fetch(args) -> int:
    load = load_blocklist(url=args.url, timeout=args.timeout, offline=args.offline)
    print(f"source:   {load.source}  ({load.origin})")
    print(f"domains:  {len(load.domains):,}")
    print(f"elapsed:  {load.elapsed_s:.2f}s")
    if load.error:
        print(f"note:     live fetch not used -> {load.error}")
    return 0


def cmd_check(args) -> int:
    load = load_blocklist(timeout=args.timeout, offline=args.offline)
    policy = build_policy(load.domains, Path(args.allowlist), Path(args.custom_blocklist), args.ho
sts_mode)
    print(f"# blocklist: {load.source}, {len(load.domains):,} domains")
    for name in args.domains:
        d = policy.decide(name)
        print(f"{name:<40} {d.action:<6} {d.rule or ''}")
    return 0


def cmd_serve(args) -> int:
    if not ipaddress.ip_address(args.host).is_loopback and not args.allow_non_loopback:
        print("refusing to bind a non-loopback address without --allow-non-loopback", file=sys.std
err)
        return 2
```

```
    host, port = parse_upstream(args.upstream)
    cfg = ResolverConfig(upstream_host=host, upstream_port=port, upstream_timeout=args.upstream_timeout)
    sh = build_sinkhole(cfg, db_path=args.db, offline=args.offline, fetch_timeout=args.timeout,
                        allowlist=Path(args.allowlist), custom_blocklist=Path(args.custom_blocklist),
                        hosts_mode=args.hosts_mode, log_clients=not args.no_client_ip)
    srv = start_dns_server(sh.resolver, host=args.host, port=args.port, tcp=True)
    print(f"blocklist: {sh.load.source} ({len(sh.load.domains):,} domains); allowlist rules: {len(sh.policy.allowlist)}")
    print(f"listening on {srv.host}:{srv.port} (UDP+TCP) -> upstream {host}:{port}; log {args.db}")
    print(f"try: dig @{srv.host} -p {srv.port} ad.doubleclick.net   (Ctrl-C to stop)")
    stop = {"flag": False}
    signal.signal(signal.SIGTERM, lambda *_: stop.update(flag=True))
    deadline = time.time() + args.duration if args.duration else None
    try:
        while not stop["flag"] and (deadline is None or time.time() < deadline):
            time.sleep(0.2)
    except KeyboardInterrupt:
        pass
    finally:
        srv.stop()
        print(json.dumps(sh.querylog.stats(top_n=5)["by_action"]))
        sh.querylog.close()
    return 0


def cmd_query(args) -> int:
    try:
        reply = dns_query(args.name, args.type, server=args.server, port=args.port,
                          timeout=args.timeout, tcp=args.tcp)
    except Exception as exc:
        print(f"query failed: {type(exc).__name__}: {exc}", file=sys.stderr)
        return 1
    print(f"status: {RCODE.get(reply.header.rcode)}  id: {reply.header.id}")
    for rr in reply.rr:
        print(f"  {rr.rname}  ttl={rr.ttl}  {QTYPE.get(rr.rtype)}  {rr.rdata}")
    return 0


def cmd_stats(args) -> int:
    if not Path(args.db).exists():
        print(f"no query log at {args.db}", file=sys.stderr)
        return 1
    stats = stats_from_file(args.db, top_n=args.top)
    if args.json:
        print(json.dumps(stats, indent=2))
        return 0
    print(f"total queries: {stats['total_queries']}   block rate: {stats['block_rate_pct']}%")
    for action, count in stats["by_action"].items():
        lat = stats["latency"].get(action, {})
        print(f"  {action:<10} {count:>6}   avg {lat.get('avg_ms', 0):>7.2f} ms   max {lat.get('max_ms', 0):>8.2f} ms")
    print("top blocked:")
    for row in stats["top_blocked"]:
        print(f"  {row['count']:>5}  {row['qname']}")
    return 0


def cmd_demo(args) -> int:
    from src.demo import matcher_benchmark, run_demo
    load = load_blocklist(timeout=args.timeout, offline=args.offline)
    s = run_demo(load, Path(args.outdir), n_queries=args.queries, seed=args.seed)
    print(f"blocklist: {s['blocklist_source']} -> {s['blocklist_domains']:,} domains from {s['bloc
```

```
klist_origin']}")
    if s["blocklist_fetch_error"]:
        print(f"  (live fetch not used: {s['blocklist_fetch_error']})")
    print(f"policy: {s['policy_rules_total']:,} block rules, {s['allowlist_rules']} allow rules")
    print(f"\nsample answers (name, type, action, rcode, answer):")
    for line in s["sample_answers"]:
        print("  " + line)
    st = s["stats"]
    print(f"\nquery mix: {s['mix_queries']} queries in {s['mix_elapsed_s']}s ({s['mix_queries_per_s']} q/s over UDP loopback)")
    print(f"  action matched expectation: {s['mix_expected_action_matches']}/{s['mix_queries']}")
    print(f"  upstream received {s['upstream_queries_seen']} queries (blocked names never leave the box)")
    print(f"failure test: {s['servfail_queries']} queries to a dead upstream -> SERVFAIL, "
          f"avg {s['servfail_avg_ms']} ms (timeout {s['upstream_timeout_ms']} ms)")
    print(f"  blocked name during outage still answered: {s['blocked_answer_during_outage']}")
    print(f"\nlog totals: {st['total_queries']} queries, block rate {st['block_rate_pct']}%")
    for a, c in st["by_action"].items():
        print(f"  {a:<10} {c:>5}   avg {st['latency'].get(a, {}).get('avg_ms', 0)} ms")
    policy = build_policy(load.domains)
    bench = matcher_benchmark(policy, load.domains[:20000] + ["safe.example.org"] * 1000, repeat=3)
    print(f"\nmatcher: {bench['lookups_per_s']:,} policy lookups/s ({bench['us_per_lookup']} us each)")
    return 0


def main(argv=None) -> int:
    p = argparse.ArgumentParser(prog="dns-sinkhole", description="DNS sinkhole / ad & domain filter (dnslib)")
    sub = p.add_subparsers(dest="cmd", required=True)

    def list_opts(sp):
        sp.add_argument("--offline", action="store_true", help="skip the live fetch; use cache/bundled snapshot")
        sp.add_argument("--timeout", type=float, default=10.0, help="blocklist fetch timeout (s)")

    def policy_opts(sp):
        sp.add_argument("--allowlist", default=str(DEFAULT_ALLOWLIST))
        sp.add_argument("--custom-blocklist", default=str(DEFAULT_CUSTOM_BLOCKLIST))
        sp.add_argument("--hosts-mode", choices=["exact", "suffix"], default="exact",
                        help="hosts entries block exact names (default) or names + subdomains")

    sp = sub.add_parser("fetch-blocklist", help="download the StevenBlack hosts list (with fallback)")
    sp.add_argument("--url", default=STEVENBLACK_URL)
    list_opts(sp); sp.set_defaults(func=cmd_fetch)

    sp = sub.add_parser("check", help="show the policy decision for domain names (no DNS traffic)")
    sp.add_argument("domains", nargs="+")
    list_opts(sp); policy_opts(sp); sp.set_defaults(func=cmd_check)

    sp = sub.add_parser("serve", help="run the sinkhole resolver (loopback by default)")
    sp.add_argument("--host", default=DEFAULT_HOST)
    sp.add_argument("--port", type=int, default=DEFAULT_PORT)
    sp.add_argument("--upstream", default="1.1.1.1:53", help="upstream resolver IP[:port]")
    sp.add_argument("--upstream-timeout", type=float, default=2.0)
    sp.add_argument("--db", default="outputs/queries.db")
    sp.add_argument("--duration", type=float, default=0, help="stop after N seconds (0 = until Ctrl-C)")
    sp.add_argument("--no-client-ip", action="store_true", help="do not store client IPs in the log")
    sp.add_argument("--allow-non-loopback", action="store_true", help="permit binding a LAN interface")
```

```
    list_opts(sp); policy_opts(sp); sp.set_defaults(func=cmd_serve)

    sp = sub.add_parser("query", help="send one DNS query (a tiny dig)")
    sp.add_argument("name")
    sp.add_argument("--type", default="A")
    sp.add_argument("--server", default=DEFAULT_HOST)
    sp.add_argument("--port", type=int, default=DEFAULT_PORT)
    sp.add_argument("--timeout", type=float, default=3.0)
    sp.add_argument("--tcp", action="store_true")
    sp.set_defaults(func=cmd_query)

    sp = sub.add_parser("stats", help="summarise a query log database")
    sp.add_argument("--db", default="outputs/queries.db")
    sp.add_argument("--top", type=int, default=10)
    sp.add_argument("--json", action="store_true")
    sp.set_defaults(func=cmd_stats)

    sp = sub.add_parser("demo", help="self-contained loopback demo with a stub upstream")
    sp.add_argument("--outdir", default="outputs")
    sp.add_argument("--queries", type=int, default=400)
    sp.add_argument("--seed", type=int, default=7)
    list_opts(sp); sp.set_defaults(func=cmd_demo)

    args = p.parse_args(argv)
    return args.func(args)


if __name__ == "__main__":
    sys.exit(main())

```

```
"""Streamlit dashboard for the DNS sinkhole query log.

Reads a live SQLite log if one exists (outputs/queries.db from `cli.py serve`),
otherwise the bundled demo export (outputs/demo_query_log.csv). Works offline.

    streamlit run app.py
"""
import json
import sqlite3
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

from src.blocklist import FALLBACK_SNAPSHOT, DEFAULT_CACHE, parse_hosts
from src.core import build_policy

ROOT = Path(__file__).resolve().parent
LIVE_DB = ROOT / "outputs" / "queries.db"
DEMO_CSV = ROOT / "outputs" / "demo_query_log.csv"
DEMO_STATS = ROOT / "outputs" / "demo_stats.json"
COLORS = {"blocked": "#d62728", "allowed": "#2ca02c", "forwarded": "#1f77b4", "servfail": "#7f7f7f
"}

st.set_page_config(page_title="DNS Sinkhole", layout="wide")


@st.cache_data
def load_log() -> tuple[pd.DataFrame, str]:
    if LIVE_DB.exists():
        with sqlite3.connect(f"file:{LIVE_DB}?mode=ro", uri=True) as conn:
            df = pd.read_sql_query("SELECT * FROM queries", conn)
        if len(df):
            return df, f"live log {LIVE_DB.name}"
    return pd.read_csv(DEMO_CSV), "bundled demo log"


@st.cache_resource
def load_policy():
    # Offline on purpose: use the cached live list if present, else the bundled snapshot.
    src = DEFAULT_CACHE if DEFAULT_CACHE.exists() else FALLBACK_SNAPSHOT
    domains = parse_hosts(src.read_text(encoding="utf-8", errors="replace"))
    return build_policy(domains), len(domains), src.name


df, source = load_log()
df["time"] = pd.to_datetime(df["ts"], unit="s")
st.title("DNS Sinkhole dashboard")
st.caption(f"Data: {source} - {len(df):,} queries")

overview, log_tab, domains_tab, tester = st.tabs(["Overview", "Query log", "Top domains", "Policy
tester"])

with overview:
    counts = df["action"].value_counts()
    total = len(df)
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Total queries", f"{total:,}")
    c2.metric("Blocked", f"{counts.get('blocked', 0):,}", f"{100 * counts.get('blocked', 0) / max(
total, 1):.1f}%")
    c3.metric("Forwarded + allowed", f"{counts.get('forwarded', 0) + counts.get('allowed', 0):,}")
    c4.metric("SERVFAIL", f"{counts.get('servfail', 0):,}")
    left, right = st.columns(2)
    pie = px.pie(names=counts.index, values=counts.values, color=counts.index,
```

```
                 color_discrete_map=COLORS, title="Decisions")
    left.plotly_chart(pie, width="stretch")
    lat = px.box(df, x="action", y="latency_ms", color="action", color_discrete_map=COLORS,
                 log_y=True, title="Latency by decision (ms, log)")
    right.plotly_chart(lat, width="stretch")
    if DEMO_STATS.exists() and source.startswith("bundled"):
        s = json.loads(DEMO_STATS.read_text())
        st.info(f"Demo run: blocklist source = {s['blocklist_source']} ({s['blocklist_domains']:,} domains); "
                f"{s['mix_expected_action_matches']}/{s['mix_queries']} queries got the expected decision; "
                f"dead-upstream SERVFAIL avg {s['servfail_avg_ms']} ms.")

with log_tab:
    actions = st.multiselect("Decision", list(COLORS), default=list(COLORS))
    needle = st.text_input("Domain contains", "")
    view = df[df["action"].isin(actions)]
    if needle:
        view = view[view["qname"].str.contains(needle.lower(), regex=False)]
    st.dataframe(view.sort_values("ts", ascending=False)[
        ["time", "qname", "qtype", "action", "rule", "rcode", "latency_ms"]], width="stretch")

with domains_tab:
    n = st.slider("Top N", 5, 30, 10)
    blocked = df[df["action"] == "blocked"]["qname"].value_counts().head(n)
    queried = df["qname"].value_counts().head(n)
    a, b = st.columns(2)
    a.plotly_chart(px.bar(x=blocked.values, y=blocked.index, orientation="h", title="Top blocked",
                          labels={"x": "queries", "y": ""}, color_discrete_sequence=[COLORS["blocked"]]),
                   width="stretch")
    b.plotly_chart(px.bar(x=queried.values, y=queried.index, orientation="h", title="Top queried",
                          labels={"x": "queries", "y": ""}), width="stretch")

with tester:
    policy, n_domains, list_name = load_policy()
    st.write(f"Policy: {n_domains:,} hosts-list domains from `{list_name}` + custom rules + allowlist.")
    names = st.text_area("Domains to test (one per line)",
                         "ad.doubleclick.net\nnotdoubleclick.net\nwww.googleadservices.com\npixel.ads.example")
    rows = []
    for name in [n.strip() for n in names.splitlines() if n.strip()]:
        d = policy.decide(name)
        rows.append({"domain": name, "decision": d.action, "rule": d.rule or ""})
    st.table(pd.DataFrame(rows))

```

06-dns-sinkhole/src/__init__.py

```
"""DNS Sinkhole / Ad & Domain Filter.

A small, readable DNS filtering resolver built on dnslib:
policy matching (allowlist overrides blocklist), sinkholed A/AAAA answers,
upstream forwarding with timeout/SERVFAIL fallback, and SQLite query logging.
"""
__version__ = "1.0.0"

```

```
"""Loading block/allow lists: hosts-file parsing, live fetch with fallback.

The default public source is the StevenBlack unified hosts file
(https://github.com/StevenBlack/hosts, MIT licence), which lists ad, tracking
and malware hostnames in ``0.0.0.0 hostname`` form.

Fetching is defensive:
  * a hard timeout, and a size cap so a hostile/misbehaving server cannot
    exhaust memory;
  * on ANY failure (DNS, TLS, timeout, HTTP error, empty/garbage body) the
    loader falls back to a small bundled snapshot so the sinkhole still starts;
  * the result always records whether the list came from ``live`` or ``fallback``.
"""
from __future__ import annotations

import time
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable, Optional

import requests

from .domains import RuleSet, normalize_domain

STEVENBLACK_URL = "https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts"
PROJECT_ROOT = Path(__file__).resolve().parent.parent
FALLBACK_SNAPSHOT = PROJECT_ROOT / "data" / "stevenblack_snapshot.txt"
DEFAULT_CACHE = PROJECT_ROOT / "data" / "cache" / "stevenblack_hosts.txt"
MAX_DOWNLOAD_BYTES = 20 * 1024 * 1024  # 20 MB; the real file is ~2.3 MB

# Hostnames that appear in hosts files for the local machine, never blocked.
_LOCAL_NAMES = {
    "localhost", "localhost.localdomain", "local", "broadcasthost",
    "ip6-localhost", "ip6-loopback", "ip6-localnet", "ip6-mcastprefix",
    "ip6-allnodes", "ip6-allrouters", "ip6-allhosts", "0.0.0.0",
}
_SINK_ADDRESSES = {"0.0.0.0", "127.0.0.1", "::", "::1"}


def parse_hosts(text: str) -> list[str]:
    """Extract blockable hostnames from hosts-file text.

    Accepts ``<ip> <host> [host ...] [# comment]`` lines where the IP is a
    sink address (0.0.0.0 / 127.0.0.1 / :: / ::1). Lines mapping to real IPs
    are ignored on purpose: they are redirections, not blocks. Comments, blank
    lines, local machine names and invalid hostnames are skipped. Order is kept
    and duplicates removed.
    """
    seen: dict[str, None] = {}
    for raw in text.splitlines():
        line = raw.split("#", 1)[0].strip()
        if not line:
            continue
        parts = line.split()
        if len(parts) < 2 or parts[0] not in _SINK_ADDRESSES:
            continue
        for host in parts[1:]:
            if host.lower() in _LOCAL_NAMES:
                continue
            name = normalize_domain(host)
            # Require at least one dot: bare words in hosts files are local names.
            if name and "." in name:
                seen.setdefault(name, None)
    return list(seen)

```

```
66
67 def parse_rules_file(path: Path) -> RuleSet:
68     """Load a rule file (allowlist / custom blocklist) into a RuleSet."""
69     rules = RuleSet()
70     if path and Path(path).exists():
71         rules.add_rules(Path(path).read_text(encoding="utf-8").splitlines())
72     return rules
73
74
75 @dataclass
76 class BlocklistLoad:
77     domains: list[str]
78     source: str            # "live" | "cache" | "fallback"
79     origin: str            # URL or file path the list came from
80     elapsed_s: float
81     error: Optional[str] = None   # why live fetch failed, if it did
82
83
84 def fetch_hosts(url: str = STEVENBLACK_URL, timeout: float = 10.0,
85                 session: Optional[requests.Session] = None) -> str:
86     """Download a hosts file with a timeout and a size cap. Raises on failure."""
87     http = session or requests
88     headers = {"User-Agent": "dns-sinkhole-portfolio/1.0 (+educational project)"}
89     with http.get(url, timeout=timeout, headers=headers, stream=True) as resp:
90         resp.raise_for_status()
91         chunks, total = [], 0
92         for chunk in resp.iter_content(chunk_size=65536):
93             total += len(chunk)
94             if total > MAX_DOWNLOAD_BYTES:
95                 raise ValueError(f"download exceeded {MAX_DOWNLOAD_BYTES} bytes")
96             chunks.append(chunk)
97     return b"".join(chunks).decode("utf-8", errors="replace")
98
99
100 def load_blocklist(url: str = STEVENBLACK_URL, timeout: float = 10.0,
101                    cache_path: Optional[Path] = DEFAULT_CACHE,
102                    fallback_path: Path = FALLBACK_SNAPSHOT,
103                    offline: bool = False,
104                    session: Optional[requests.Session] = None) -> BlocklistLoad:
105     """Fetch the live list; on failure use the cache, then the bundled snapshot."""
106     start = time.perf_counter()
107     error = None
108     if not offline:
109         try:
110             text = fetch_hosts(url, timeout=timeout, session=session)
111             domains = parse_hosts(text)
112             if len(domains) < 10:
113                 raise ValueError(f"only {len(domains)} domains parsed; refusing a near-empty list"
    )
114             if cache_path:
115                 cache_path = Path(cache_path)
116                 cache_path.parent.mkdir(parents=True, exist_ok=True)
117                 tmp = cache_path.with_suffix(".tmp")
118                 tmp.write_text(text, encoding="utf-8")
119                 tmp.replace(cache_path)          # atomic swap
120             return BlocklistLoad(domains, "live", url, time.perf_counter() - start)
121         except Exception as exc:  # network, HTTP, TLS, size, parse
122             error = f"{type(exc).__name__}: {exc}"
123     else:
124         error = "offline mode requested"
125     if cache_path and Path(cache_path).exists():
126         domains = parse_hosts(Path(cache_path).read_text(encoding="utf-8", errors="replace"))
127         if domains:
128             return BlocklistLoad(domains, "cache", str(cache_path), time.perf_counter() - start, e
    rror)
```

```
    domains = parse_hosts(Path(fallback_path).read_text(encoding="utf-8"))
    return BlocklistLoad(domains, "fallback", str(fallback_path), time.perf_counter() - start, err
or)


def build_blocklist(domains: Iterable[str], extra_rules: Optional[RuleSet] = None,
                    hosts_mode: str = "exact") -> RuleSet:
    """Turn hostnames into a RuleSet.

    ``hosts_mode="exact"`` (default) blocks exactly the listed names, matching
    how hosts files behave. ``"suffix"`` also blocks every subdomain of each
    listed name (stricter, more false positives).
    """
    rules = RuleSet()
    target = rules.exact if hosts_mode == "exact" else rules.suffix
    target.update(domains)
    if extra_rules:
        rules.exact |= extra_rules.exact
        rules.suffix |= extra_rules.suffix
    return rules

```

06-dns-sinkhole/src/core.py

```
"""Wire the pieces together: lists -> policy -> resolver -> query log."""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Optional

from .blocklist import (BlocklistLoad, PROJECT_ROOT, build_blocklist, load_blocklist,
                        parse_rules_file)
from .domains import DomainPolicy
from .querylog import QueryLog
from .resolver import ResolverConfig, SinkholeResolver

DEFAULT_ALLOWLIST = PROJECT_ROOT / "data" / "allowlist.txt"
DEFAULT_CUSTOM_BLOCKLIST = PROJECT_ROOT / "data" / "custom_blocklist.txt"


@dataclass
class Sinkhole:
    resolver: SinkholeResolver
    policy: DomainPolicy
    querylog: QueryLog
    load: Optional[BlocklistLoad]


def build_policy(domains: list[str], allowlist: Optional[Path] = DEFAULT_ALLOWLIST,
                 custom_blocklist: Optional[Path] = DEFAULT_CUSTOM_BLOCKLIST,
                 hosts_mode: str = "exact") -> DomainPolicy:
    extra = parse_rules_file(custom_blocklist) if custom_blocklist else None
    return DomainPolicy(build_blocklist(domains, extra, hosts_mode),
                        parse_rules_file(allowlist) if allowlist else None)


def build_sinkhole(config: ResolverConfig, db_path: str | Path = ":memory:",
                   offline: bool = False, fetch_timeout: float = 10.0,
                   allowlist: Optional[Path] = DEFAULT_ALLOWLIST,
                   custom_blocklist: Optional[Path] = DEFAULT_CUSTOM_BLOCKLIST,
                   hosts_mode: str = "exact", log_clients: bool = True,
                   load: Optional[BlocklistLoad] = None) -> Sinkhole:
    load = load or load_blocklist(timeout=fetch_timeout, offline=offline)
    policy = build_policy(load.domains, allowlist, custom_blocklist, hosts_mode)
    qlog = QueryLog(db_path, log_clients=log_clients)
    return Sinkhole(SinkholeResolver(policy, config, qlog), policy, qlog, load)

```

```
"""Reproducible end-to-end demo on loopback only.

Starts two DNS servers on random free 127.0.0.1 ports:

    demo client  --UDP-->  sinkhole (SinkholeResolver)  --UDP-->  stub upstream
                                    |                               (StubZoneResolver,
                                    +--> SQLite query log            documentation IPs only)

then sends a seeded mix of queries (real blocklist hostnames, allowlisted
names, label-boundary look-alikes, normal names, NXDOMAIN names), kills the
upstream to show the SERVFAIL path, and writes stats/exports. Nothing is
intercepted and no system resolver settings are touched.
"""
from __future__ import annotations

import csv
import json
import random
import sqlite3
import time
from pathlib import Path
from typing import Optional

from dnslib import QTYPE, RCODE

from .blocklist import BlocklistLoad
from .core import build_sinkhole
from .resolver import ResolverConfig
from .server import StubZoneResolver, blackhole_udp_socket, dns_query, start_dns_server

# Stub upstream zone: RFC 5737 / RFC 3849 documentation addresses only.
STUB_RECORDS = {
    "example.com":              {"A": ["192.0.2.10"], "AAAA": ["2001:db8::10"]},
    "www.example.com":          {"A": ["192.0.2.11"], "AAAA": ["2001:db8::11"]},
    "docs.python.org":          {"A": ["192.0.2.20"], "AAAA": ["2001:db8::20"]},
    "github.com":               {"A": ["192.0.2.30"]},
    "www.googleadservices.com": {"A": ["192.0.2.40"]},        # allowlisted
    "news.ads.example":         {"A": ["198.51.100.5"]},      # allowlisted under *.ads.example
    "myads.example":            {"A": ["198.51.100.6"]},      # boundary: NOT *.ads.example
    "notdoubleclick.net":       {"A": ["198.51.100.7"]},      # boundary: not doubleclick.net
    "doubleclick.net.example.org": {"A": ["198.51.100.8"]},   # blocked name as a prefix only
}
NORMAL = ["example.com", "www.example.com", "docs.python.org", "github.com"]
BOUNDARY = ["myads.example", "notdoubleclick.net", "doubleclick.net.example.org"]
ALLOWED = ["www.googleadservices.com", "news.ads.example"]
CUSTOM_BLOCKED = ["pixel.ads.example", "a.b.ads.example", "tracker.test", "v2.telemetry.test"]
NXDOMAIN_NAMES = ["no-such-host.example.net", "typo-gihtub.com"]


def build_query_mix(blocklist_domains: list[str], n: int = 400, seed: int = 7) -> list[tuple[str,
str, str]]:
    """Return [(name, qtype, expected_action)] with a Zipf-like popularity skew."""
    rng = random.Random(seed)
    candidates = sorted(set(blocklist_domains) - set(ALLOWED))
    popular_blocked = rng.sample(candidates, k=min(30, len(candidates)))
    weights = [1.0 / (i + 1) for i in range(len(popular_blocked))]
    mix = []
    for _ in range(n):
        r = rng.random()
        qtype = "AAAA" if rng.random() < 0.3 else "A"
        if r < 0.38:
            mix.append((rng.choices(popular_blocked, weights)[0], qtype, "blocked"))
        elif r < 0.46:
            mix.append((rng.choice(CUSTOM_BLOCKED), qtype, "blocked"))
        elif r < 0.78:
```

```
            mix.append((rng.choice(NORMAL), qtype, "forwarded"))
        elif r < 0.86:
            mix.append((rng.choice(BOUNDARY), "A", "forwarded"))
        elif r < 0.94:
            mix.append((rng.choice(ALLOWED), "A", "allowed"))
        else:
            mix.append((rng.choice(NXDOMAIN_NAMES), "A", "forwarded"))
    return mix


def run_demo(load: BlocklistLoad, outdir: Path, n_queries: int = 400, seed: int = 7,
             upstream_timeout: float = 0.5, n_failure: int = 10,
             db_name: str = "demo_queries.db") -> dict:
    outdir = Path(outdir)
    outdir.mkdir(parents=True, exist_ok=True)
    db_path = outdir / db_name
    if db_path.exists():
        db_path.unlink()
    for suffix in ("-wal", "-shm"):
        Path(str(db_path) + suffix).unlink(missing_ok=True)

    stub = StubZoneResolver(records=STUB_RECORDS)
    upstream = start_dns_server(stub, port=0, tcp=False)
    cfg = ResolverConfig(upstream_host="127.0.0.1", upstream_port=upstream.port,
                         upstream_timeout=upstream_timeout)
    sh = build_sinkhole(cfg, db_path=db_path, load=load, log_clients=False)
    sink = start_dns_server(sh.resolver, port=0, tcp=True)
    lines: list[str] = []
    correct = 0
    mismatches = []
    try:
        mix = build_query_mix(load.domains, n=n_queries, seed=seed)
        t0 = time.perf_counter()
        for name, qtype, expected in mix:
            reply = dns_query(name, qtype, port=sink.port, timeout=3)
            action = sh.querylog.rows(1)[0]["action"]
            correct += action == expected
            if action != expected:
                mismatches.append((name, qtype, expected, action))
        elapsed = time.perf_counter() - t0

        # Show a few individual answers verbatim for the README.
        for name, qtype in [("ad.doubleclick.net", "A"), ("ad.doubleclick.net", "AAAA"),
                            ("pixel.ads.example", "A"), ("news.ads.example", "A"),
                            ("myads.example", "A"), ("www.example.com", "AAAA"),
                            ("no-such-host.example.net", "A")]:
            reply = dns_query(name, qtype, port=sink.port, timeout=3)
            row = sh.querylog.rows(1)[0]
            answers = ", ".join(str(rr.rdata) for rr in reply.rr) or "-"
            lines.append(f"{name:<28} {qtype:<5} {row['action']:<10} {RCODE.get(reply.header.rcode
):<9} {answers}"
                         + (f"   (rule {row['rule']})" if row["rule"] else ""))

        # Failure path: point the sinkhole at an upstream that never answers.
        dead = blackhole_udp_socket()
        cfg.upstream_port = dead.getsockname()[1]
        fail_times = []
        try:
            for i in range(n_failure):
                t = time.perf_counter()
                reply = dns_query(NORMAL[i % len(NORMAL)], "A", port=sink.port, timeout=5)
                fail_times.append((time.perf_counter() - t) * 1000)
                assert reply.header.rcode == RCODE.SERVFAIL
            # Blocked names keep working while upstream is down (answered locally).
            reply = dns_query("ad.doubleclick.net", "A", port=sink.port, timeout=3)
```

```
            blocked_during_outage = str(reply.rr[0].rdata) if reply.rr else None
        finally:
            dead.close()
        stats = sh.querylog.stats(top_n=10)
    finally:
        sink.stop()
        upstream.stop()

    summary = {
        "blocklist_source": load.source,
        "blocklist_origin": load.origin,
        "blocklist_domains": len(load.domains),
        "blocklist_fetch_error": load.error,
        "policy_rules_total": len(sh.policy.blocklist),
        "allowlist_rules": len(sh.policy.allowlist),
        "mix_queries": len(mix),
        "mix_expected_action_matches": correct,
        "mix_mismatches": mismatches,
        "mix_elapsed_s": round(elapsed, 3),
        "mix_queries_per_s": round(len(mix) / elapsed, 1),
        "upstream_queries_seen": stub.queries_seen,
        "servfail_queries": n_failure,
        "servfail_avg_ms": round(sum(fail_times) / len(fail_times), 1),
        "servfail_max_ms": round(max(fail_times), 1),
        "upstream_timeout_ms": int(upstream_timeout * 1000),
        "blocked_answer_during_outage": blocked_during_outage,
        "sample_answers": lines,
        "stats": stats,
    }
    sh.querylog.close()
    (outdir / "demo_stats.json").write_text(json.dumps(summary, indent=2))
    export_csv(db_path, outdir / "demo_query_log.csv")
    return summary


def export_csv(db_path: Path, csv_path: Path) -> int:
    conn = sqlite3.connect(str(db_path))
    try:
        cur = conn.execute("SELECT ts, client, qname, qtype, action, rule, rcode, latency_ms"
                           " FROM queries ORDER BY id")
        rows = cur.fetchall()
        with open(csv_path, "w", newline="") as fh:
            wr = csv.writer(fh)
            wr.writerow([c[0] for c in cur.description])
            wr.writerows(rows)
        return len(rows)
    finally:
        conn.close()


def matcher_benchmark(policy, names: list[str], repeat: int = 5) -> dict:
    """Pure policy lookups per second (no network)."""
    t = time.perf_counter()
    for _ in range(repeat):
        for n in names:
            policy.decide(n)
    elapsed = time.perf_counter() - t
    total = repeat * len(names)
    return {"lookups": total, "elapsed_s": round(elapsed, 3),
            "lookups_per_s": int(total / elapsed), "us_per_lookup": round(elapsed / total * 1e6, 2
)}

```

06-dns-sinkhole/src/domains.py

```
"""Domain normalisation and allow/block policy matching.

Rule syntax (one rule per line, used by allowlists and custom blocklists):

    example.com        exact match only            (example.com)
    *.example.com      the domain AND every subdomain (example.com, a.example.com, a.b.example.com)
    .example.com       same as *.example.com

Matching is done on whole DNS labels, never on raw string suffixes, so a rule
for ``*.example.com`` does NOT match ``badexample.com`` or ``example.com.evil.net``.

Lookup cost is O(number of labels in the query name): the name is walked from
the full name towards the TLD and each candidate is a set membership test.
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Iterable, Optional

# RFC 1035 limits: label <= 63 octets, full name <= 253 characters (text form).
_LABEL_RE = re.compile(r"^(?!-)[a-z0-9_-]{1,63}(?<!-)$")
MAX_NAME_LEN = 253


def normalize_domain(name: str) -> Optional[str]:
    """Lower-case, strip whitespace and the trailing root dot, validate labels.

    Returns the normalised name, or ``None`` if the input is not a valid
    hostname (empty, too long, bad characters, empty label such as ``a..b``).
    Internationalised names are converted to their ASCII (punycode) form.
    """
    if not name:
        return None
    name = name.strip().rstrip(".").lower()
    if not name:
        return None
    try:
        # Convert Unicode labels (e.g. "bücher.de") to punycode ("xn--bcher-kva.de").
        name = name.encode("idna").decode("ascii") if not name.isascii() else name
    except UnicodeError:
        return None
    if len(name) > MAX_NAME_LEN:
        return None
    labels = name.split(".")
    if any(not _LABEL_RE.match(label) for label in labels):
        return None
    return name


def parent_domains(name: str) -> list[str]:
    """Return the name and every parent at a label boundary.

parent_domains("a.b.example.com")
    ['a.b.example.com', 'b.example.com', 'example.com', 'com']
    """
    labels = name.split(".")
    return [".".join(labels[i:]) for i in range(len(labels))]


@dataclass
class RuleSet:
    """A set of exact-match and suffix-match (domain + subdomains) rules."""

```

```
    exact: set[str] = field(default_factory=set)
    suffix: set[str] = field(default_factory=set)

    def add_rule(self, rule: str) -> bool:
        """Parse one rule line. Returns True if a rule was added."""
        rule = rule.split("#", 1)[0].strip()
        if not rule:
            return False
        is_suffix = False
        if rule.startswith("*."):
            rule, is_suffix = rule[2:], True
        elif rule.startswith("."):
            rule, is_suffix = rule[1:], True
        name = normalize_domain(rule)
        if name is None:
            return False
        (self.suffix if is_suffix else self.exact).add(name)
        return True

    def add_rules(self, rules: Iterable[str]) -> int:
        return sum(1 for r in rules if self.add_rule(r))

    def match(self, name: str) -> Optional[str]:
        """Return the rule that matches ``name`` (already normalised), or None.

        Exact rules are checked first, then suffix rules from the most
        specific parent to the least specific.
        """
        if name in self.exact:
            return name
        for candidate in parent_domains(name):
            if candidate in self.suffix:
                return "*." + candidate
        return None

    def __len__(self) -> int:
        return len(self.exact) + len(self.suffix)


@dataclass(frozen=True)
class Decision:
    """Outcome of a policy check for one query name."""

    action: str               # "block" | "allow" (allowlist hit) | "pass" (no rule)
    rule: Optional[str] = None


class DomainPolicy:
    """Allowlist-overrides-blocklist policy.

    Order of evaluation:
      1. If any allowlist rule matches -> ``allow`` (always forwarded, even if
         a blocklist rule also matches; this is how false positives are fixed).
      2. Else if any blocklist rule matches -> ``block`` (sinkholed).
      3. Else -> ``pass`` (forwarded upstream).
    """

    def __init__(self, blocklist: Optional[RuleSet] = None, allowlist: Optional[RuleSet] = None):
        self.blocklist = blocklist or RuleSet()
        self.allowlist = allowlist or RuleSet()

    def decide(self, qname: str) -> Decision:
        name = normalize_domain(qname)
        if name is None:
            # Malformed names are never sinkholed by policy; upstream decides.
```

06-dns-sinkhole/src/domains.py (continued)

```
            return Decision("pass")
        allow_rule = self.allowlist.match(name)
        if allow_rule:
            return Decision("allow", allow_rule)
        block_rule = self.blocklist.match(name)
        if block_rule:
            return Decision("block", block_rule)
        return Decision("pass")

```

```
"""SQLite query log and statistics.

Every query the sinkhole answers is written as one row. The DNS server
handles requests on several threads, so writes go through a single
connection guarded by a lock (SQLite in WAL mode).

Privacy: client IPs can be dropped (``log_clients=False``) because a DNS
log is effectively a browsing history for everyone on the network.
"""
from __future__ import annotations

import sqlite3
import threading
import time
from pathlib import Path
from typing import Optional

SCHEMA = """
CREATE TABLE IF NOT EXISTS queries (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    ts          REAL    NOT NULL,          -- unix time (seconds)
    client      TEXT,                      -- client IP, or NULL if not logged
    qname       TEXT    NOT NULL,
    qtype       TEXT    NOT NULL,
    action      TEXT    NOT NULL,          -- blocked | allowed | forwarded | servfail
    rule        TEXT,                      -- matching rule for blocked/allowed
    rcode       TEXT    NOT NULL,          -- NOERROR | NXDOMAIN | SERVFAIL | ...
    latency_ms  REAL    NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_queries_ts ON queries(ts);
CREATE INDEX IF NOT EXISTS idx_queries_action ON queries(action);
"""

ACTIONS = ("blocked", "allowed", "forwarded", "servfail")


class QueryLog:
    def __init__(self, path: str | Path = ":memory:", log_clients: bool = True):
        self.path = str(path)
        if self.path != ":memory:":
            Path(self.path).parent.mkdir(parents=True, exist_ok=True)
        self.log_clients = log_clients
        self._lock = threading.Lock()
        self._conn = sqlite3.connect(self.path, check_same_thread=False)
        if self.path != ":memory:":
            self._conn.execute("PRAGMA journal_mode=WAL")
        self._conn.executescript(SCHEMA)
        self._conn.commit()

    def record(self, qname: str, qtype: str, action: str, rcode: str,
               latency_ms: float, client: Optional[str] = None,
               rule: Optional[str] = None, ts: Optional[float] = None) -> None:
        if action not in ACTIONS:
            raise ValueError(f"unknown action {action!r}")
        with self._lock:
            self._conn.execute(
                "INSERT INTO queries (ts, client, qname, qtype, action, rule, rcode, latency_ms)"
                " VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
                (ts if ts is not None else time.time(),
                 client if self.log_clients else None,
                 qname, qtype, action, rule, rcode, round(latency_ms, 3)),
            )
            self._conn.commit()

    def rows(self, limit: int = 100) -> list[dict]:
```

```
        with self._lock:
            cur = self._conn.execute(
                "SELECT ts, client, qname, qtype, action, rule, rcode, latency_ms"
                " FROM queries ORDER BY id DESC LIMIT ?", (limit,))
            cols = [c[0] for c in cur.description]
            return [dict(zip(cols, r)) for r in cur.fetchall()]

    def stats(self, top_n: int = 10) -> dict:
        return compute_stats(self._conn, self._lock, top_n)

    def close(self) -> None:
        with self._lock:
            self._conn.close()


def compute_stats(conn: sqlite3.Connection, lock: Optional[threading.Lock] = None,
                  top_n: int = 10) -> dict:
    """Aggregate statistics straight from SQL."""
    lock = lock or threading.Lock()
    with lock:
        q = conn.execute
        total = q("SELECT COUNT(*) FROM queries").fetchone()[0]
        by_action = {a: 0 for a in ACTIONS}
        by_action.update(dict(q("SELECT action, COUNT(*) FROM queries GROUP BY action").fetchall()
))
        by_qtype = dict(q("SELECT qtype, COUNT(*) FROM queries GROUP BY qtype ORDER BY 2 DESC").fe
tchall())
        latency = {
            a: {"avg_ms": round(avg, 3), "max_ms": round(mx, 3)}
            for a, avg, mx in q("SELECT action, AVG(latency_ms), MAX(latency_ms)"
                                " FROM queries GROUP BY action").fetchall()
        }
        top_blocked = q("SELECT qname, COUNT(*) FROM queries WHERE action='blocked'"
                        " GROUP BY qname ORDER BY 2 DESC, 1 LIMIT ?", (top_n,)).fetchall()
        top_queried = q("SELECT qname, COUNT(*) FROM queries"
                        " GROUP BY qname ORDER BY 2 DESC, 1 LIMIT ?", (top_n,)).fetchall()
        top_clients = q("SELECT client, COUNT(*) FROM queries WHERE client IS NOT NULL"
                        " GROUP BY client ORDER BY 2 DESC LIMIT ?", (top_n,)).fetchall()
    return {
        "total_queries": total,
        "by_action": by_action,
        "block_rate_pct": round(100.0 * by_action["blocked"] / total, 2) if total else 0.0,
        "by_qtype": by_qtype,
        "latency": latency,
        "top_blocked": [{"qname": n, "count": c} for n, c in top_blocked],
        "top_queried": [{"qname": n, "count": c} for n, c in top_queried],
        "top_clients": [{"client": n, "count": c} for n, c in top_clients],
    }


def stats_from_file(path: str | Path, top_n: int = 10) -> dict:
    """Open a log database read-only and compute statistics."""
    conn = sqlite3.connect(f"file:{path}?mode=ro", uri=True)
    try:
        return compute_stats(conn, None, top_n)
    finally:
        conn.close()

```

```
"""The sinkhole resolver: policy check, sinkhole answers, upstream forwarding.

Request flow for each query::

    client --> [SinkholeResolver.resolve]
                    |
                    +-- policy.decide(qname)
                    |      block --> A: 0.0.0.0 / AAAA: ::  (other types: NXDOMAIN)
                    |      allow --> forward (allowlist beats blocklist)
                    |      pass  --> forward
                    |
                    +-- forward: raw UDP to upstream with timeout
                    |      reply id must match -> return upstream answer as-is
                    |      timeout / socket error / bad reply -> SERVFAIL
                    |
                    +-- QueryLog.record(...)  (SQLite)
"""
from __future__ import annotations

import socket
import time
from dataclasses import dataclass
from typing import Optional

from dnslib import AAAA, QTYPE, RCODE, RR, A, DNSRecord
from dnslib.server import BaseResolver

from .domains import DomainPolicy
from .querylog import QueryLog


@dataclass
class ResolverConfig:
    upstream_host: str = "1.1.1.1"
    upstream_port: int = 53
    upstream_timeout: float = 2.0        # seconds before giving up -> SERVFAIL
    sinkhole_ipv4: str = "0.0.0.0"
    sinkhole_ipv6: str = "::"
    block_ttl: int = 60                  # short TTL so un-blocking takes effect quickly
    blocked_other_rcode: str = "NXDOMAIN"  # answer for blocked non-A/AAAA queries


class SinkholeResolver(BaseResolver):
    """dnslib resolver implementing the sinkhole policy."""

    def __init__(self, policy: DomainPolicy, config: Optional[ResolverConfig] = None,
                 querylog: Optional[QueryLog] = None):
        self.policy = policy
        self.config = config or ResolverConfig()
        self.querylog = querylog

    # dnslib calls resolve(request, handler) for every query (UDP and TCP).
    def resolve(self, request: DNSRecord, handler) -> DNSRecord:
        start = time.perf_counter()
        qname = str(request.q.qname).rstrip(".")
        qtype = QTYPE.get(request.q.qtype, str(request.q.qtype))
        decision = self.policy.decide(qname)

        if decision.action == "block":
            reply = self._sinkhole_reply(request)
            action = "blocked"
        else:
            reply, ok = self.forward(request)
            action = "servfail" if not ok else ("allowed" if decision.action == "allow" else "forw
arded")
```

```
65
66          latency_ms = (time.perf_counter() - start) * 1000.0
67          if self.querylog is not None:
68              client = None
69              if handler is not None and getattr(handler, "client_address", None):
70                  client = handler.client_address[0]
71              self.querylog.record(qname=qname, qtype=qtype, action=action,
72                                   rcode=RCODE.get(reply.header.rcode), latency_ms=latency_ms,
73                                   client=client, rule=decision.rule)
74          return reply
75
76      def _sinkhole_reply(self, request: DNSRecord) -> DNSRecord:
77          """Answer a blocked query locally without contacting upstream."""
78          reply = request.reply()          # copies id + question, sets QR/RA flags
79          qname, qtype = request.q.qname, request.q.qtype
80          ttl = self.config.block_ttl
81          if qtype == QTYPE.A:
82              reply.add_answer(RR(qname, QTYPE.A, ttl=ttl, rdata=A(self.config.sinkhole_ipv4)))
83          elif qtype == QTYPE.AAAA:
84              reply.add_answer(RR(qname, QTYPE.AAAA, ttl=ttl, rdata=AAAA(self.config.sinkhole_ipv6))
    )
85          else:
86              reply.header.rcode = getattr(RCODE, self.config.blocked_other_rcode)
87          return reply
88
89      def forward(self, request: DNSRecord) -> tuple[DNSRecord, bool]:
90          """Send the query to upstream over UDP.
91
92          Returns ``(reply, ok)``. On timeout, socket error or an unparseable /
93          mismatched reply, ``ok`` is False and the reply is a local SERVFAIL, so
94          the client gets a fast, definite failure instead of hanging.
95          """
96          cfg = self.config
97          try:
98              raw = request.send(cfg.upstream_host, cfg.upstream_port, timeout=cfg.upstream_timeout)
99              reply = DNSRecord.parse(raw)
100             if reply.header.id != request.header.id:
101                 raise ValueError("upstream reply id mismatch")
102             if reply.header.tc:
103                 # Truncated over UDP: retry the same query over TCP.
104                 raw = request.send(cfg.upstream_host, cfg.upstream_port, tcp=True,
105                                    timeout=cfg.upstream_timeout)
106                 reply = DNSRecord.parse(raw)
107             return reply, True
108         except (socket.timeout, OSError, ValueError, Exception):  # noqa: B014 - any failure -> SE
    RVFAIL
109             reply = request.reply()
110             reply.header.rcode = RCODE.SERVFAIL
111             return reply, False
112
```

06-dns-sinkhole/src/server.py

```
"""Start/stop helpers around dnslib's threaded UDP/TCP DNSServer.

Servers bind to 127.0.0.1 and a high port by default. Port 0 asks the OS for
any free port, which is what the tests use so they never collide or need root.
"""
from __future__ import annotations

import socket
from dataclasses import dataclass, field
from typing import Optional

from dnslib import AAAA, QTYPE, RCODE, RR, A, DNSRecord
from dnslib.server import BaseResolver, DNSLogger, DNSServer

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 53535            # unprivileged; avoids 53 (root) and 5353 (mDNS)


def quiet_logger() -> DNSLogger:
    """A dnslib logger that prints nothing (we log to SQLite instead)."""
    return DNSLogger(log="-request,-reply,-truncated,-error,-recv,-send,-data", prefix=False)


@dataclass
class RunningServer:
    host: str
    port: int
    udp: DNSServer
    tcp: Optional[DNSServer] = None

    def stop(self) -> None:
        for srv in (self.udp, self.tcp):
            if srv is not None:
                srv.stop()
                srv.server.server_close()


def start_dns_server(resolver: BaseResolver, host: str = DEFAULT_HOST, port: int = 0,
                     tcp: bool = True, logger: Optional[DNSLogger] = None) -> RunningServer:
    """Start UDP (and optionally TCP) servers in background threads.

    With ``port=0`` the UDP socket gets a free port from the OS and TCP is
    bound to the same number (retrying a few times if it happens to be taken).
    """
    logger = logger or quiet_logger()
    last_error: Optional[Exception] = None
    for _ in range(20):
        udp = DNSServer(resolver, address=host, port=port, logger=logger)
        real_port = udp.server.server_address[1]
        tcp_srv = None
        if tcp:
            try:
                tcp_srv = DNSServer(resolver, address=host, port=real_port, tcp=True, logger=logge
r)
            except OSError as exc:      # TCP port taken: try another UDP port
                udp.server.server_close()
                last_error = exc
                if port:                # a fixed port was requested; don't loop
                    raise
                continue
        udp.start_thread()
        if tcp_srv:
            tcp_srv.start_thread()
        return RunningServer(host, real_port, udp, tcp_srv)
    raise RuntimeError(f"could not bind UDP+TCP on the same port: {last_error}")
```

```
65
66
67  # ---------------------------------------------------------------------------
68  # Local stub upstream: an authoritative-only toy server used by tests and the
69  # offline demo, so nothing depends on (or touches) the real internet.
70  # ---------------------------------------------------------------------------
71  @dataclass
72  class StubZoneResolver(BaseResolver):
73      """Answers A/AAAA from a dict; NXDOMAIN for unknown names."""
74
75      records: dict = field(default_factory=dict)   # name -> {"A": [...], "AAAA": [...]}
76      ttl: int = 300
77      queries_seen: int = 0
78
79      def resolve(self, request: DNSRecord, handler) -> DNSRecord:
80          self.queries_seen += 1
81          reply = request.reply()
82          name = str(request.q.qname).rstrip(".").lower()
83          qtype = QTYPE.get(request.q.qtype)
84          zone = self.records.get(name)
85          if zone is None:
86              reply.header.rcode = RCODE.NXDOMAIN
87              return reply
88          for value in zone.get(qtype, []):
89              rdata = A(value) if qtype == "A" else AAAA(value)
90              reply.add_answer(RR(request.q.qname, getattr(QTYPE, qtype), ttl=self.ttl, rdata=rdata)
    )
91          return reply
92
93
94  def blackhole_udp_socket(host: str = DEFAULT_HOST) -> socket.socket:
95      """A bound UDP socket that never answers: simulates a dead upstream."""
96      sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
97      sock.bind((host, 0))
98      return sock
99
100
101 def free_closed_port(host: str = DEFAULT_HOST) -> int:
102     """A port number that had nothing listening a moment ago (ICMP-refused upstream)."""
103     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
104     sock.bind((host, 0))
105     port = sock.getsockname()[1]
106     sock.close()
107     return port
108
109
110 def dns_query(name: str, qtype: str = "A", server: str = DEFAULT_HOST,
111               port: int = DEFAULT_PORT, timeout: float = 3.0, tcp: bool = False) -> DNSRecord:
112     """Send one query to a DNS server and return the parsed reply."""
113     q = DNSRecord.question(name, qtype)
114     raw = q.send(server, port, tcp=tcp, timeout=timeout)
115     return DNSRecord.parse(raw)
116
```

06-dns-sinkhole/tests/conftest.py

```
"""Shared fixtures: ephemeral loopback DNS servers, always shut down."""
import pytest

from src.domains import DomainPolicy, RuleSet
from src.querylog import QueryLog
from src.resolver import ResolverConfig, SinkholeResolver
from src.server import StubZoneResolver, start_dns_server

STUB = {
    "example.com": {"A": ["192.0.2.10"], "AAAA": ["2001:db8::10"]},
    "safe.example.org": {"A": ["192.0.2.20"]},
    "notads.example": {"A": ["198.51.100.1"]},
    "badexample.com": {"A": ["198.51.100.2"]},
    "news.ads.example": {"A": ["198.51.100.3"]},
    "tracker.test.example.org": {"A": ["198.51.100.4"]},
}


@pytest.fixture
def stub_upstream():
    stub = StubZoneResolver(records=STUB)
    srv = start_dns_server(stub, port=0, tcp=False)
    try:
        yield stub, srv
    finally:
        srv.stop()


def make_policy():
    block = RuleSet()
    block.add_rules(["ads.example.com", "*.ads.example", "tracker.test", "*.example.com.evil.test"
])
    allow = RuleSet()
    allow.add_rules(["news.ads.example"])
    return DomainPolicy(block, allow)


@pytest.fixture
def sinkhole(stub_upstream):
    """A running sinkhole on a random port, forwarding to the stub upstream."""
    stub, up = stub_upstream
    qlog = QueryLog(":memory:")
    cfg = ResolverConfig(upstream_host="127.0.0.1", upstream_port=up.port, upstream_timeout=0.5)
    resolver = SinkholeResolver(make_policy(), cfg, qlog)
    srv = start_dns_server(resolver, port=0, tcp=True)
    try:
        yield {"server": srv, "resolver": resolver, "log": qlog, "stub": stub, "cfg": cfg}
    finally:
        srv.stop()
        qlog.close()

```

06-dns-sinkhole/tests/test_blocklist.py

```
import pytest
import requests

from src import blocklist as bl

HOSTS = """
# comment line
127.0.0.1 localhost
::1 ip6-localhost
0.0.0.0 0.0.0.0
0.0.0.0 ads.example.com tracker.example.net   # two hosts on one line
127.0.0.1 Metrics.Example.ORG
0.0.0.0 ads.example.com
93.184.216.34 redirect.example.com
0.0.0.0 not..valid
0.0.0.0 barehost
"""


def test_parse_hosts_extracts_only_sinkholed_valid_names():
    assert bl.parse_hosts(HOSTS) == ["ads.example.com", "tracker.example.net", "metrics.example.org"]


def test_bundled_fallback_snapshot_parses():
    domains = bl.parse_hosts(bl.FALLBACK_SNAPSHOT.read_text())
    assert len(domains) >= 2000
    assert "ad.doubleclick.net" in domains


class _Resp:
    def __init__(self, body: bytes, status=200):
        self.body, self.status_code = body, status
    def __enter__(self): return self
    def __exit__(self, *a): return False
    def raise_for_status(self):
        if self.status_code >= 400:
            raise requests.HTTPError(f"HTTP {self.status_code}")
    def iter_content(self, chunk_size=65536):
        for i in range(0, len(self.body), chunk_size):
            yield self.body[i:i + chunk_size]


class _Session:
    def __init__(self, resp=None, exc=None):
        self.resp, self.exc, self.calls = resp, exc, []
    def get(self, url, **kw):
        self.calls.append((url, kw))
        if self.exc:
            raise self.exc
        return self.resp


def test_live_fetch_success_uses_timeout_and_caches(tmp_path):
    body = "\n".join(f"0.0.0.0 host{i}.example.com" for i in range(50)).encode()
    sess = _Session(_Resp(body))
    cache = tmp_path / "cache.txt"
    load = bl.load_blocklist(timeout=3, cache_path=cache, session=sess)
    assert load.source == "live" and len(load.domains) == 50
    assert sess.calls[0][1]["timeout"] == 3
    assert cache.exists()


def test_timeout_falls_back_to_bundled_snapshot(tmp_path):
    sess = _Session(exc=requests.Timeout("simulated timeout"))
```

```
    load = bl.load_blocklist(timeout=0.1, cache_path=tmp_path / "none.txt", session=sess)
    assert load.source == "fallback" and "Timeout" in load.error
    assert len(load.domains) >= 2000


def test_http_error_uses_previous_cache_first(tmp_path):
    cache = tmp_path / "cache.txt"
    cache.write_text("0.0.0.0 cached1.example.com\n0.0.0.0 cached2.example.com\n")
    load = bl.load_blocklist(cache_path=cache, session=_Session(_Resp(b"", 503)))
    assert load.source == "cache" and load.domains == ["cached1.example.com", "cached2.example.com
"]


def test_near_empty_download_is_rejected(tmp_path):
    load = bl.load_blocklist(cache_path=tmp_path / "c.txt", session=_Session(_Resp(b"<html>oops</h
tml>")))
    assert load.source == "fallback" and "refusing" in load.error


def test_oversized_download_is_cut_off(tmp_path, monkeypatch):
    monkeypatch.setattr(bl, "MAX_DOWNLOAD_BYTES", 1000)
    load = bl.load_blocklist(cache_path=tmp_path / "c.txt", session=_Session(_Resp(b"0" * 5000)))
    assert load.source == "fallback" and "exceeded" in load.error


def test_hosts_mode_suffix_blocks_subdomains():
    exact = bl.build_blocklist(["ads.example.com"], hosts_mode="exact")
    suffix = bl.build_blocklist(["ads.example.com"], hosts_mode="suffix")
    assert exact.match("x.ads.example.com") is None
    assert suffix.match("x.ads.example.com") == "*.ads.example.com"


@pytest.mark.network
def test_live_stevenblack_fetch_or_honest_fallback(tmp_path):
    load = bl.load_blocklist(timeout=15, cache_path=tmp_path / "c.txt")
    # Either the real list arrived (tens of thousands of names) or we said why not.
    assert (load.source == "live" and len(load.domains) > 10000) or load.error

```

06-dns-sinkhole/tests/test_domains.py

```python
import pytest

from src.domains import DomainPolicy, RuleSet, normalize_domain, parent_domains


@pytest.mark.parametrize("raw,expected", [
    ("Example.COM.", "example.com"),
    ("  ads.example.com  ", "ads.example.com"),
    ("bücher.de", "xn--bcher-kva.de"),
    ("_dmarc.example.com", "_dmarc.example.com"),
])
def test_normalize_valid(raw, expected):
    assert normalize_domain(raw) == expected


@pytest.mark.parametrize("raw", ["", ".", "a..b", "-bad.com", "bad-.com", "sp ace.com", "a" * 64 + ".com",
                                 ("a" * 60 + ".") * 5 + "com"])
def test_normalize_invalid(raw):
    assert normalize_domain(raw) is None


def test_parent_domains_walks_label_boundaries():
    assert parent_domains("a.b.example.com") == ["a.b.example.com", "b.example.com", "example.com", "com"]


def test_exact_rule_matches_only_that_name():
    rs = RuleSet(); rs.add_rule("ads.example.com")
    assert rs.match("ads.example.com") == "ads.example.com"
    assert rs.match("x.ads.example.com") is None
    assert rs.match("example.com") is None


@pytest.mark.parametrize("rule", ["*.ads.example", ".ads.example"])
def test_suffix_rule_matches_domain_and_subdomains(rule):
    rs = RuleSet(); rs.add_rule(rule)
    for name in ["ads.example", "pixel.ads.example", "a.b.c.ads.example"]:
        assert rs.match(name) == "*.ads.example"


@pytest.mark.parametrize("name", ["badads.example", "notads.example", "ads.example.evil.test", "ads.examples"])
def test_suffix_rule_respects_label_boundary(name):
    rs = RuleSet(); rs.add_rule("*.ads.example")
    assert rs.match(name) is None


def test_rule_parsing_skips_comments_and_garbage():
    rs = RuleSet()
    added = rs.add_rules(["# comment", "", "good.example  # trailing", "bad..name", "*.zone.test"])
    assert added == 2 and "good.example" in rs.exact and "zone.test" in rs.suffix


def test_allowlist_overrides_blocklist_exact_and_suffix():
    block = RuleSet(); block.add_rules(["*.ads.example", "tracker.test"])
    allow = RuleSet(); allow.add_rules(["news.ads.example", "*.tracker.test"])
    policy = DomainPolicy(block, allow)
    assert policy.decide("pixel.ads.example").action == "block"
    assert policy.decide("news.ads.example").action == "allow"
    assert policy.decide("tracker.test").action == "allow"       # allow suffix covers apex
    assert policy.decide("NEWS.ADS.EXAMPLE.").action == "allow"  # case + trailing dot
    assert policy.decide("other.example").action == "pass"

```

06-dns-sinkhole/tests/test_domains.py (continued)

```
62
63 def test_malformed_query_name_is_passed_not_blocked():
64     block = RuleSet(); block.add_rule("*.example")
65     assert DomainPolicy(block).decide("bad..example").action == "pass"
66
```

06-dns-sinkhole/tests/test_querylog_cli.py

```python
import subprocess
import sys
from pathlib import Path

import pytest

from src.querylog import QueryLog, stats_from_file
from src.server import start_dns_server
from src.resolver import SinkholeResolver
from src.domains import DomainPolicy, RuleSet

ROOT = Path(__file__).resolve().parent.parent


def test_querylog_file_stats_and_privacy(tmp_path):
    db = tmp_path / "q.db"
    log = QueryLog(db, log_clients=False)
    log.record("a.example", "A", "blocked", "NOERROR", 0.1, client="10.0.0.5", rule="a.example")
    log.record("b.example", "AAAA", "forwarded", "NOERROR", 4.0, client="10.0.0.5")
    log.record("b.example", "A", "servfail", "SERVFAIL", 500.0)
    log.close()
    st = stats_from_file(db)
    assert st["total_queries"] == 3 and st["by_qtype"] == {"A": 2, "AAAA": 1}
    assert st["top_clients"] == []          # client IPs were not stored
    assert st["latency"]["servfail"]["max_ms"] == 500.0


def test_querylog_rejects_unknown_action():
    with pytest.raises(ValueError):
        QueryLog().record("x.example", "A", "maybe", "NOERROR", 1.0)


def run_cli(*args, timeout=60):
    return subprocess.run([sys.executable, "cli.py", *args], cwd=ROOT, capture_output=True,
                          text=True, timeout=timeout)


def test_cli_help_lists_subcommands():
    out = run_cli("--help").stdout
    for cmd in ["fetch-blocklist", "check", "serve", "query", "stats", "demo"]:
        assert cmd in out


def test_cli_check_offline_uses_fallback_and_allowlist():
    out = run_cli("check", "--offline", "ad.doubleclick.net", "www.googleadservices.com",
                  "notdoubleclick.net").stdout
    lines = {l.split()[0]: l.split()[1] for l in out.splitlines() if not l.startswith("#")}
    assert lines == {"ad.doubleclick.net": "block", "www.googleadservices.com": "allow",
                     "notdoubleclick.net": "pass"}


def test_cli_serve_refuses_non_loopback_without_flag():
    r = run_cli("serve", "--host", "0.0.0.0", "--offline")
    assert r.returncode == 2 and "non-loopback" in r.stderr


def test_cli_query_against_ephemeral_sinkhole():
    block = RuleSet(); block.add_rule("*.ads.example")
    srv = start_dns_server(SinkholeResolver(DomainPolicy(block)), port=0)
    try:
        r = run_cli("query", "x.ads.example", "--type", "AAAA", "--port", str(srv.port))
    finally:
        srv.stop()
    assert "status: NOERROR" in r.stdout and "AAAA  ::" in r.stdout

```

06-dns-sinkhole/tests/test_server.py

```python
"""Integration tests over real UDP/TCP sockets on 127.0.0.1 (random ports)."""
import time

import pytest

from dnslib import QTYPE, RCODE

from src.server import blackhole_udp_socket, dns_query, free_closed_port


def ask(sh, name, qtype="A", tcp=False):
    return dns_query(name, qtype, port=sh["server"].port, timeout=3, tcp=tcp)


def last_action(sh):
    return sh["log"].rows(1)[0]["action"]


def test_blocked_a_query_returns_sinkhole_ipv4(sinkhole):
    r = ask(sinkhole, "ads.example.com", "A")
    assert r.header.rcode == RCODE.NOERROR
    assert [str(rr.rdata) for rr in r.rr] == ["0.0.0.0"]
    assert r.rr[0].ttl == 60 and last_action(sinkhole) == "blocked"


def test_blocked_aaaa_query_returns_sinkhole_ipv6(sinkhole):
    r = ask(sinkhole, "pixel.ads.example", "AAAA")
    assert r.rr[0].rtype == QTYPE.AAAA and str(r.rr[0].rdata) == "::"


def test_blocked_other_qtype_gets_nxdomain(sinkhole):
    r = ask(sinkhole, "tracker.test", "MX")
    assert r.header.rcode == RCODE.NXDOMAIN and not r.rr


def test_blocked_queries_never_reach_upstream(sinkhole):
    before = sinkhole["stub"].queries_seen
    for _ in range(5):
        ask(sinkhole, "ads.example.com")
    assert sinkhole["stub"].queries_seen == before


def test_allowlisted_name_under_blocked_suffix_is_forwarded(sinkhole):
    r = ask(sinkhole, "news.ads.example")
    assert str(r.rr[0].rdata) == "198.51.100.3" and last_action(sinkhole) == "allowed"


def test_label_boundary_lookalikes_are_not_blocked(sinkhole):
    for name, ip in [("notads.example", "198.51.100.1"), ("badexample.com", "198.51.100.2"),
                     ("tracker.test.example.org", "198.51.100.4")]:
        r = ask(sinkhole, name)
        assert str(r.rr[0].rdata) == ip, name
        assert last_action(sinkhole) == "forwarded"


def test_forwarding_returns_upstream_answers_a_and_aaaa(sinkhole):
    assert str(ask(sinkhole, "example.com", "A").rr[0].rdata) == "192.0.2.10"
    assert str(ask(sinkhole, "example.com", "AAAA").rr[0].rdata) == "2001:db8::10"


def test_upstream_nxdomain_is_passed_through(sinkhole):
    r = ask(sinkhole, "missing.example.net")
    assert r.header.rcode == RCODE.NXDOMAIN and last_action(sinkhole) == "forwarded"


```

```
def test_reply_id_matches_query_and_tcp_works(sinkhole):
    from dnslib import DNSRecord
    q = DNSRecord.question("example.com")
    r = DNSRecord.parse(q.send("127.0.0.1", sinkhole["server"].port, timeout=3))
    assert r.header.id == q.header.id
    assert str(ask(sinkhole, "ads.example.com", tcp=True).rr[0].rdata) == "0.0.0.0"
    assert str(ask(sinkhole, "example.com", tcp=True).rr[0].rdata) == "192.0.2.10"


def test_silent_upstream_times_out_to_servfail(sinkhole):
    dead = blackhole_udp_socket()
    try:
        sinkhole["cfg"].upstream_port = dead.getsockname()[1]
        t = time.perf_counter()
        r = ask(sinkhole, "example.com")
        elapsed = time.perf_counter() - t
    finally:
        dead.close()
    assert r.header.rcode == RCODE.SERVFAIL
    assert 0.4 <= elapsed < 2.0                # bounded by upstream_timeout=0.5s
    assert last_action(sinkhole) == "servfail"


def test_refused_upstream_port_gives_servfail_fast(sinkhole):
    sinkhole["cfg"].upstream_port = free_closed_port()
    t = time.perf_counter()
    r = ask(sinkhole, "example.com")
    assert r.header.rcode == RCODE.SERVFAIL and time.perf_counter() - t < 2.0


def test_blocking_still_works_when_upstream_is_down(sinkhole):
    sinkhole["cfg"].upstream_port = free_closed_port()
    assert str(ask(sinkhole, "ads.example.com").rr[0].rdata) == "0.0.0.0"


def test_query_log_counts_and_stats(sinkhole):
    for name in ["ads.example.com", "ads.example.com", "pixel.ads.example", "example.com", "news.a
ds.example"]:
        ask(sinkhole, name)
    st = sinkhole["log"].stats()
    assert st["total_queries"] == 5
    assert st["by_action"] == {"blocked": 3, "allowed": 1, "forwarded": 1, "servfail": 0}
    assert st["block_rate_pct"] == 60.0
    assert st["top_blocked"][0] == {"qname": "ads.example.com", "count": 2}
    assert st["top_clients"][0]["client"] == "127.0.0.1"


@pytest.mark.network
def test_real_public_upstream_forwarding():
    """Forward one real query through the sinkhole to a public resolver (1.1.1.1)."""
    from src.domains import DomainPolicy, RuleSet
    from src.resolver import ResolverConfig, SinkholeResolver
    from src.server import start_dns_server
    block = RuleSet(); block.add_rule("ad.doubleclick.net")
    res = SinkholeResolver(DomainPolicy(block), ResolverConfig("1.1.1.1", 53, upstream_timeout=3))
    srv = start_dns_server(res, port=0)
    try:
        r = dns_query("example.com", "A", port=srv.port, timeout=5)
        b = dns_query("ad.doubleclick.net", "A", port=srv.port, timeout=5)
    finally:
        srv.stop()
    assert r.header.rcode == RCODE.NOERROR and any(rr.rtype == QTYPE.A for rr in r.rr)
    assert str(b.rr[0].rdata) == "0.0.0.0"

```

06-dns-sinkhole/.gitignore

```
__pycache__/
*.pyc
.pytest_cache/
*.db
*.db-wal
*.db-shm
data/cache/
.venv/
venv/
.streamlit/secrets.toml

```

06-dns-sinkhole/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
timeout = 60
markers =
    network: needs internet access (live blocklist fetch / public upstream)

```

06-dns-sinkhole/scripts/make_charts.py

```
"""Render README charts from the demo's real query log (outputs/demo_query_log.csv)."""
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd

OUT = Path(__file__).resolve().parent.parent / "outputs"
COLORS = {"blocked": "#d62728", "allowed": "#2ca02c", "forwarded": "#1f77b4", "servfail": "#7f7f7f
"}


def main() -> None:
    df = pd.read_csv(OUT / "demo_query_log.csv")
    order = [a for a in COLORS if a in set(df.action)]

    counts = df.action.value_counts().reindex(order)
    fig, ax = plt.subplots(figsize=(7, 4))
    bars = ax.bar(counts.index, counts.values, color=[COLORS[a] for a in counts.index])
    ax.bar_label(bars)
    ax.set_title(f"Sinkhole decisions for {len(df)} real loopback queries")
    ax.set_ylabel("queries")
    fig.tight_layout(); fig.savefig(OUT / "chart_actions.png", dpi=120); plt.close(fig)

    fig, ax = plt.subplots(figsize=(7, 4))
    data = [df.loc[df.action == a, "latency_ms"] for a in order]
    ax.boxplot(data, tick_labels=order, showfliers=True)
    ax.set_yscale("log")
    ax.set_ylabel("resolver latency (ms, log scale)")
    ax.set_title("Latency: local sinkhole answers vs forwarded vs upstream timeout")
    fig.tight_layout(); fig.savefig(OUT / "chart_latency.png", dpi=120); plt.close(fig)

    top = df[df.action == "blocked"].qname.value_counts().head(12)[::-1]
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.barh(top.index, top.values, color=COLORS["blocked"])
    ax.set_title("Top blocked domains in the demo query mix")
    ax.set_xlabel("queries")
    fig.tight_layout(); fig.savefig(OUT / "chart_top_blocked.png", dpi=120); plt.close(fig)
    print("charts written to", OUT)


if __name__ == "__main__":
    main()

```